



ThreadStock Product & Engineering Blueprint

AI Inventory & Commerce OS for Fashion

Product Blueprint v1.1

17 September 2026

ThreadStock Developer Blueprint Pack

Start here

Included agent instruction files

Product principle

Non-negotiables

ThreadStock Architecture

1. Architecture goals
2. Recommended stack
3. Repository shape
4. Feature layering
5. Multi-tenant hierarchy
6. Inventory architecture
7. Transaction boundaries
8. Offline architecture
9. Routing architecture
10. AI architecture
11. Integration architecture
12. Design system architecture
13. Observability
14. Environments
15. Architectural decision rule

ThreadStock Product Blueprint

Top-level navigation

Navigation rules

Onboarding

Overview

Global

Inventory

Sales

Purchasing

Transfers

AI Studio

Automations

Insights

Settings

Help

Core end-to-end journeys

Page behavior contract

ThreadStock Detailed Module Specifications

Overview

Inventory

Sales

Purchasing

Transfers

AI Studio

Automations

Insights

Settings

Onboarding

Global surfaces

ThreadStock Behavior Specification

1. Global behavior

2. Authentication and workspace entry

3. Inventory

4. Sales

5. Purchasing

6. Transfers

7. AI Studio

8. Automations

9. Insights

10. Search and command palette

11. Offline and sync

12. Permissions

13. Accessibility and localization

Canonical Route Manifest

Route naming rules

ThreadStock Canonical Data Model

Identity and tenancy

Catalog

Inventory

Sales

Purchasing

Transfers

AI

Automations

Insights and reports

Platform

Invariants

ThreadStock State Machines

Purchase order

Transfer

Stock count

Sale

AI action proposal

Automation

Import job

Integration sync

ThreadStock AI System

Product role

Capability tiers

Proposal lifecycle

Server boundary

Tool policy

Freshness

User experience

AI history

Automations

Evaluation

ThreadStock Security and Authorization Contract

1. Trust boundaries

2. Tenant isolation

3. Permission model

4. Sensitive operations

5. Secrets

6. RLS

7. Server commands

8. Audit

9. Files

10. Payments

11. Offline data

12. Logs and analytics

13. Security definition of done

ThreadStock Visual Identity Contract

1. Brand character

2. Canonical brand assets

3. Color system

4. Typography

5. Background and surface treatment

6. Surface geometry

7. Primary actions
8. Navigation
9. AI visual language
10. Charts and data visualization
11. Form and onboarding style
12. Flutter implementation contract
13. Visual QA checklist

ThreadStock Design System Engineering Contract

Product character
Canonical visual tokens
Typography
Brand assets
Background behavior
Layout rules
Spacing and geometry
Primary actions
Components
AI visual language
Motion
Accessibility
Internationalization

ThreadStock Testing Strategy

- Goals
1. Static gates
 2. Domain unit tests
 3. Data/repository tests
 4. Database tests
 5. Widget tests
 6. Integration journeys
 7. Golden/visual testing
 8. Performance
 9. Regression policy

ThreadStock Implementation Roadmap

Phase 0 - Foundation
Phase 1 - Catalog and inventory core
Phase 2 - Sales
Phase 3 - Purchasing and receiving
Phase 4 - Transfers and stock counts
Phase 5 - Insights foundation
Phase 6 - AI read intelligence
Phase 7 - AI prepared actions and approvals

Phase 8 - Automations

Phase 9 - Integrations and enterprise settings

Phase 10 - Production hardening

Delivery rule

ThreadStock Definition of Done

Product

UX

Domain/data

Security

Quality

Documentation

AI Coding Platform Instructions

Cursor

Google Antigravity

VS Code / GitHub Copilot

Rule precedence philosophy

First prompt for a coding agent

Visual implementation setup

ThreadStock Engineering Skills

1. Product workflow skill

2. Flutter feature skill

3. Responsive/adaptive UI skill

4. Supabase/Postgres skill

5. Inventory accounting skill

6. Offline/sync skill

7. AI workflow skill

8. Authorization skill

9. Testing skill

10. Migration skill

11. Observability skill

12. Product copy skill

Visual identity update v1.1. This edition adopts the approved ThreadStock light-first Atelier visual direction: warm silk/ivory surfaces, near-black ink, antique gold, Cormorant Garamond editorial

display typography and Inter operational UI typography. The included logo, silk background and approved reference screenshots are implementation references for all coding agents.

Core product principle: ThreadStock should do more work than the user. Prefer detect → recommend → prepare → review → approve → execute → audit.

ThreadStock Developer Blueprint Pack

ThreadStock is a premium AI-powered fashion inventory and commerce operating system for Android, iOS, Windows and macOS. This pack is the engineering source-of-truth for AI coding agents and human developers.

Start here

1. **AGENTS.md** - universal project instructions for AI coding agents.
2. **ARCHITECTURE.md** - production architecture and module boundaries.
3. **PRODUCT_BLUEPRINT.md** - pages, subpages, ownership and navigation.
4. **ROUTE_MANIFEST.md** - canonical routes. Do not invent competing routes.
5. **BEHAVIOR_SPEC.md** - interaction, workflow and state behavior.
6. **DATA_MODEL.md** - canonical domain entities and data invariants.
7. **AI_SYSTEM.md** - AI recommendation, preparation, approval and execution architecture.
8. **SECURITY.md** - RBAC, RLS, auditability and sensitive-action rules.
9. **VISUAL_IDENTITY.md** - canonical palette, fonts, logo/background assets and approved visual references.
10. **DESIGN_SYSTEM.md** - component, layout, responsive, motion and accessibility rules.
11. **TESTING_STRATEGY.md** - test pyramid and critical journeys.
12. **IMPLEMENTATION_ROADMAP.md** - recommended build sequence.
13. **DEFINITION_OF_DONE.md** - shipping gates.
14. **PLATFORM_INSTRUCTIONS.md** - Cursor, Antigravity and VS Code setup.

Included agent instruction files

- Cursor: `.cursor/rules/*.mdc`
- Google Antigravity workspace rules: `.agents/rules/*.md`
- VS Code / GitHub Copilot: `.github/copilot-instructions.md` plus `.github/instructions/*.instructions.md`
- Cross-agent: `AGENTS.md`

Product principle

ThreadStock should do more work than the user. Prefer **detect** → **recommend** → **prepare** → **review** → **approve** → **execute** → **audit** over forcing users through repetitive manual workflows.

Non-negotiables

- Do not implement fake buttons, placeholder actions or dead-end screens.
- Do not bypass permission checks or Supabase RLS.
- Do not put model/API secrets in Flutter clients.
- Do not allow AI to execute sensitive actions without the configured approval boundary.
- Do not silently overwrite stock during offline conflict resolution.
- Do not create a second route, entity or status vocabulary when one already exists here.
- Desktop and mobile share domain logic but have intentionally different layouts.

ThreadStock Architecture

1. Architecture goals

ThreadStock must support a boutique with one phone and a multinational fashion business with many stores without creating two products. The architecture therefore prioritizes:

- multi-tenant isolation;
- multi-location inventory correctness;
- offline-tolerant operations;
- auditable stock and money movement;
- progressive feature complexity;
- shared domain behavior across Android, iOS, Windows and macOS;
- AI that can prepare real actions safely;
- testability and replaceable infrastructure adapters.

2. Recommended stack

Client

- Flutter: one codebase, adaptive presentation.
- State/DI: Riverpod-style provider architecture with explicit controller/view-model state. Do not hide business workflows in widget state.
- Routing: typed route definitions with guards and canonical paths from [ROUTE_MANIFEST.md](#).
- Local persistence: relational SQLite/Drift-style local database for cache, outbox and offline read models.
- Secure storage: platform secure storage for session/device-sensitive tokens only.
- Scanner/camera, printing and share functionality behind platform adapters.

Backend

- Supabase Auth for identity/session lifecycle.
- Postgres for authoritative operational data.
- RLS for tenant/location/role enforcement.
- Storage for product images, documents and approved attachments.
- Realtime only where it improves operational freshness; do not depend on realtime for correctness.
- Edge Functions/server services for privileged integrations, AI gateway, webhooks and secret-bearing operations.

- Postgres RPC/functions for atomic business operations such as sale completion, receiving, transfer state transitions and stock reconciliation.

3. Repository shape

```
threadstock/  
  lib/  
    app/  
      bootstrap/  
      router/  
      shell/  
      l10n/  
    core/  
      auth/  
      errors/  
      logging/  
      connectivity/  
      sync/  
      permissions/  
      platform/  
    design_system/  
      tokens/  
      components/  
      responsive/  
    domain_shared/  
      money/  
      quantity/  
      identifiers/  
      pagination/  
    features/  
      overview/  
      inventory/  
      sales/  
      purchasing/  
      transfers/  
      ai_studio/  
      automations/  
      insights/  
      settings/  
      onboarding/  
      global_search/  
      approvals/  
      notifications/  
  supabase/  
    migrations/  
    functions/  
    tests/  
    seed/  
  test/  
  integration_test/  
  docs/  
  tools/  
  .cursor/rules/  
  .agents/rules/  
  .github/instructions/
```

Start as a single Flutter application with feature boundaries. Split internal packages only when a boundary has proven independent build/test/versioning value. Avoid premature package fragmentation.

4. Feature layering

Each feature can use:

```
presentation -> domain -> repository interface -> data adapter -> Supabase/local store
```

Rules:

- Presentation depends on domain, not Supabase DTOs.
- Domain never imports Flutter widgets.
- Data adapters translate external schemas into domain models.
- Domain services own business rules that cross simple entity methods.
- Controllers coordinate UI state and use cases, but do not become god objects.

5. Multi-tenant hierarchy

Canonical scope:

```
business -> locations -> operational records
```

A user accesses a business through a membership. Membership maps to one or more roles/scopes. A role contains permissions. Location restrictions are additive to permission rules.

Important: `business_id` is part of every operational aggregate or is derivable through an immutable foreign-key chain. Never trust a client-supplied business ID without server validation.

6. Inventory architecture

Use two concepts:

1. `inventory_ledger`: append-only events explaining why quantity changed.
2. `inventory_balance`: materialized/current balance optimized for reads.

Every stock-changing transaction must write the ledger and update/recompute balances atomically. The balance is never the only audit evidence.

Recommended balance dimensions:

- business_id
- location_id
- variant_id
- available_qty
- committed_qty
- incoming_qty (may be derived from open supply/transfer records)
- damaged_qty
- version

Avoid a single ambiguous `quantity` column for all business meaning.

7. Transaction boundaries

Use server-side transactional operations for:

- complete sale;
- refund/return that changes stock and payment records;
- receive purchase order;
- dispatch transfer;
- receive transfer;
- approve stock-count reconciliation;
- create a purchase order from an AI proposal;
- execute automation-created actions.

Each command accepts an idempotency/client operation key and returns the canonical server result.

8. Offline architecture

Offline capability should be selective, not pretend-everything-is-offline.

Safe candidates:

- catalog lookup from cache;
- barcode lookup if cached;
- draft sale or configured offline sale;
- stock count capture;
- receiving capture where policy allows;
- notes/drafts.

Server-authoritative or connection-required candidates may include high-risk approvals, integration changes, role changes and actions requiring fresh financial state.

Outbox

Queued operations are immutable envelopes. On sync, the server either:

- accepts exactly once;
- returns already-processed canonical result;
- rejects with a business conflict;
- rejects with authorization/session error.

Do not retry destructive operations without idempotency.

9. Routing architecture

Use one canonical route vocabulary from [ROUTE_MANIFEST.md](#). Desktop and mobile can render different pages/shells for the same domain destination. Avoid parallel route trees such as `/desktop/inventory` and `/mobile/inventory` unless a platform-only workflow truly requires it.

Route guards run in this order:

1. app bootstrap/environment valid;
2. authenticated session;
3. active business membership;
4. onboarding/business setup state;
5. required permission;
6. required location scope;
7. feature flag/capability if applicable.

10. AI architecture

AI client UI talks to a ThreadStock AI gateway, not directly to model providers with privileged keys.

```
Flutter -> AI Gateway -> policy/context builder -> model/tool orchestration
                                     -> read tools
                                     -> proposal tools
                                     -> approved execution tools
```

AI reads are scoped to the actor. Write-like tool calls create proposals unless a configured safe automation permits execution. See [AI_SYSTEM.md](#).

11. Integration architecture

External channels (Shopify, accounting, payments, shipping) are adapters. Store:

- integration connection metadata;
- encrypted/secret credentials server-side;
- sync cursor/checkpoints;
- external-to-internal ID mapping;
- last success/error;
- webhook delivery/audit.

Never model an external platform's schema as the internal ThreadStock domain model.

12. Design system architecture

Keep a shared Flutter design system with semantic tokens, not feature-specific hex values. Use adaptive components where behavior differs by platform:

- desktop data table vs mobile entity list;
- desktop right inspector vs mobile bottom sheet/page;
- desktop sidebar vs mobile bottom navigation;
- desktop hover/focus vs mobile press/haptic states.

See [DESIGN_SYSTEM.md](#).

13. Observability

Track business operation outcomes with correlation IDs. Key events include:

- auth/session failures;
- sync conflicts;
- sale completion/refund;
- receiving/transfer transitions;
- stock reconciliation;
- AI proposal lifecycle;
- automation runs;
- integration sync failures.

Separate developer diagnostics from user-visible error copy.

14. Environments

Recommended:

- local development;
- staging;
- production.

Each has separate Supabase projects/credentials and storage. Never use production service-role secrets locally. Migrations promote in sequence and are applied by controlled tooling/CI, not ad hoc UI edits.

15. Architectural decision rule

When uncertain, optimize in this order:

1. correctness of stock/money/security;
2. user workflow clarity;
3. auditability and recoverability;
4. maintainability/testability;
5. performance;
6. cleverness.

ThreadStock Product Blueprint

This document is the canonical information architecture and page ownership map. Routes are defined in `ROUTE_MANIFEST.md`.

Top-level navigation

Desktop: Overview → Inventory → Sales → Purchasing → Transfers → AI Studio → Automations → Insights → Settings

Mobile primary: Home → Inventory → Scan → Sales → AI

Mobile secondary modules are exposed through More, Home shortcuts and contextual flows. They do not become an oversized bottom navigation.

Navigation rules

- A subpage belongs to one primary domain even when it can be deep-linked from another domain.
- Opening an entity from search/notification should deep-link to the entity, not the module home.
- Back navigation should restore filters, scroll position and selection when practical.
- Desktop quick review uses a right inspector; full page opens for editing or deeper work.
- Mobile quick review uses bottom sheets where appropriate; substantial work gets a dedicated page.
- Every route must have loading, empty (where relevant), error, permission and safe-back behavior.

Onboarding

Page	Route	Enter from	Permission	Purpose
Welcome	/onboarding/welcome	Entry	Public	Start business setup or demo workspace
Business setup	/onboarding/business	/onboarding/welcome	Public	Business identity, country, language, currency
Location setup	/onboarding/location	/onboarding/business	Public	Create first store/warehouse
Commerce setup	/onboarding/commerce	/onboarding/location	Public	Sales channels, payment methods, tax mode

Page	Route	Enter from	Permission	Purpose
Inventory starting point	<code>/onboarding/inventory</code>	<code>/onboarding/commerce</code>	<code>Public</code>	Start fresh, import, or connect store
Team setup	<code>/onboarding/team</code>	<code>/onboarding/inventory</code>	<code>Public</code>	Invite initial users with role + location
Setup complete	<code>/onboarding/complete</code>	<code>/onboarding/team</code>	<code>Public</code>	Review setup and enter workspace

Overview

Page	Route	Enter from	Permission	Purpose
Overview dashboard	<code>/overview</code>	App root	<code>Authenticated</code>	Business pulse, attention queue, AI insight, quick actions

Global

Page	Route	Enter from	Permission	Purpose
Notification center	<code>/notifications</code>	Header bell	<code>Authenticated</code>	Operational, AI and system notifications
Approval center	<code>/approvals</code>	Header / Overview / AI	<code>Permission: approvals.view</code>	PO, transfer, adjustment, refund and AI approvals
Recent activity	<code>/activity</code>	Header / entity activity	<code>Authenticated</code>	Cross-module chronological operational activity
Sync activity	<code>/sync</code>	Profile / offline banner	<code>Authenticated</code>	Offline queue, retries, conflicts
My profile	<code>/profile</code>	Profile menu	<code>Authenticated</code>	Personal profile, security, preferences

Inventory

Page	Route	Enter from	Permission	Purpose
Inventory Studio	<code>/inventory</code>	Sidebar	<code>inventory.view</code>	Products/styles, search, filters, bulk actions
Create product	<code>/inventory/products/new</code>	<code>/inventory</code>	<code>inventory.product.create</code>	Create style, attributes, commerce and stock defaults
Product detail	<code>/inventory/products/:productId</code>	<code>/inventory</code>	<code>inventory.view</code>	Overview, variants, stock, purchasing, sales, activity
Edit product	<code>/inventory/products/:productId/edit</code>	Product detail	<code>inventory.product.edit</code>	Edit style and commerce data

Page	Route	Enter from	Permission	Purpose
Variant matrix	/inventory/products/:productId/variants	Product detail	inventory.product.edit	Color x size/SKU matrix
Product stock	/inventory/products/:productId/stock	Product detail	inventory.view	Location stock, incoming, committed, coverage
Product activity	/inventory/products/:productId/activity	Product detail	inventory.view	Ledger timeline for one product/variant
Stock counts	/inventory/counts	Inventory tabs	inventory.count.view	Count list and status
Start stock count	/inventory/counts/new	/inventory/counts	inventory.count.create	Location/scope/counting mode
Active stock count	/inventory/counts/:countId	/inventory/counts	inventory.count.execute	Scan/manual count progress
Count reconciliation	/inventory/counts/:countId/reconcile	Count detail	inventory.count.approve	Review variances and post adjustments
Adjustment history	/inventory/adjustments	Inventory tabs	inventory.adjustment.view	Manual adjustments and audit
Stock adjustment	/inventory/adjustments/new	Product / Inventory	inventory.adjustment.create	Adjust quantity with reason and audit
Barcode & labels	/inventory/labels	Inventory tabs	inventory.labels.print	Generate/print labels
Categories	/inventory/catalog/categories	Catalog setup	inventory.catalog.manage	Category hierarchy
Collections	/inventory/catalog/collections	Catalog setup	inventory.catalog.manage	Seasons and collections
Inventory import	/inventory/import	Inventory actions	inventory.import	Upload/map/validate/import CSV/XLSX
Import review	/inventory/import/:jobId	/inventory/import	inventory.import	Mapping, validation and final review

Sales

Page	Route	Enter from	Permission	Purpose
Sales overview	/sales	Sidebar	sales.view	Today, recent transactions and sales operations
Quick sale	/sales/new	/sales	sales.create	POS cart, customer, discount, payment

Page	Route	Enter from	Permission	Purpose
Held sales	/sales/held	Sales tabs	sales.create	Resume/cancel paused sales
Sale detail	/sales/:saleId	/sales	sales.view	Items, payment, customer, receipt, activity
Receipt	/sales/:saleId/receipt	Sale detail	sales.view	Print/share/download receipt
Return / exchange	/sales/:saleId/return	Sale detail	sales.return	Return/refund/exchange flow
Customers	/sales/customers	Sales tabs	customer.view	Customer directory
Create customer	/sales/customers/new	Customers / New sale	customer.create	Lightweight customer profile
Customer detail	/sales/customers/:customerId	Customers	customer.view	Purchases, returns, preferences, activity

Purchasing

Page	Route	Enter from	Permission	Purpose
Purchase orders	/purchasing	Sidebar	purchasing.view	PO list, incoming stock, spend
Create purchase order	/purchasing/orders/new	/purchasing	purchasing.po.create	Supplier, products, quantities, costs
Purchase order detail	/purchasing/orders/:poId	/purchasing	purchasing.view	Items, receiving, costs, documents, activity
Edit purchase order	/purchasing/orders/:poId/edit	PO detail	purchasing.po.edit	Edit draft/allowed PO fields
Receive purchase order	/purchasing/orders/:poId/receive	PO detail / Receiving	purchasing.receive	Full/partial/damaged receiving
Suppliers	/purchasing/suppliers	Purchasing tabs	supplier.view	Supplier directory and performance
Create supplier	/purchasing/suppliers/new	Suppliers	supplier.create	Supplier profile and terms
Supplier detail	/purchasing/suppliers/:supplierId	Suppliers	supplier.view	Products, POs, lead time, activity
Receiving queue	/purchasing/receiving	Purchasing tabs	purchasing.receive	Expected/in-transit/partial/recent receipts
Return to supplier	/purchasing/returns/new	PO detail	purchasing.return	Return received stock and expected credit

Transfers

Page	Route	Enter from	Permission	Purpose
Transfers	/transfers	Sidebar	transfer.view	Incoming/outgoing/ history
Create transfer	/transfers/new	/transfers	transfer.create	Source, destination, products and quantities
Transfer detail	/ transfers/:transferId	/transfers	transfer.view	Items, shipment, activity, status
Dispatch transfer	/ transfers/:transferId/dispatch	Transfer detail	transfer.dispatch	Pick/confirm and dispatch stock
Receive transfer	/ transfers/:transferId/receive	Transfer detail / Incoming	transfer.receive	Receive and record discrepancies

AI Studio

Page	Route	Enter from	Permission	Purpose
Intelligence home	/ai	Sidebar / mobile AI	ai.use	Proactive insights + natural language input
AI action queue	/ai/actions	AI tabs / approvals	ai.action.view	Prepared proposals awaiting review
AI action detail	/ai/ actions/:actionId	AI actions	ai.action.view	What/why/impact/ confidence/edit/approve
AI history	/ai/history	AI tabs	ai.history.view	Conversation, recommendations, reports and actions
Forecast center	/ai/forecasts	AI tabs	ai.forecast.view	Demand forecast overview
Forecast detail	/ai/ forecasts/:variantId	Forecast center	ai.forecast.view	Variant/location trajectory and action
Anomaly center	/ai/anomalies	AI tabs	ai.anomaly.view	Unusual activity needing review
Anomaly investigation	/ai/ anomalies/:anomalyId	Anomaly center	ai.anomaly.view	Context and safe next actions

Automations

Page	Route	Enter from	Permission	Purpose
Automations	/automations	Sidebar	automation.view	Active/paused/history

Page	Route	Enter from	Permission	Purpose
Create automation	/automations/new	/automations	automation.create	Natural language + transparent rule builder
Automation detail	/automations/:automationId	/automations	automation.view	Rules, health, scope, run stats
Edit automation	/automations/:automationId/edit	Automation detail	automation.edit	Edit trigger/conditions/actions/approval
Run history	/automations/runs	Automations tabs	automation.view	Run outcomes and approvals
Run detail	/automations/runs/:runId	Run history	automation.view	Business-readable result/failure detail

Insights

Page	Route	Enter from	Permission	Purpose
Insights home	/insights	Sidebar	insights.view	Executive summary and drilldowns
Sales analytics	/insights/sales	Insights tabs	insights.sales	Revenue, units, AOV, returns, margin
Inventory analytics	/insights/inventory	Insights tabs	insights.inventory	Value, coverage, sell-through, risk
Dead stock	/insights/dead-stock	Inventory analytics	insights.inventory	No/low movement inventory and suggested actions
Stock ageing	/insights/stock-ageing	Inventory analytics	insights.inventory	Age buckets and working capital
Profitability	/insights/profitability	Insights tabs	insights.profitability	Revenue, COGS, gross profit and margin
Location comparison	/insights/locations	Insights tabs	insights.locations	Store/warehouse comparison
Supplier performance	/insights/suppliers	Insights tabs	insights.suppliers	Lead time, fill rate, spend, cost trend
Forecast accuracy	/insights/forecast-accuracy	Insights tabs	insights.forecast	Forecast vs actual
Report Studio	/insights/reports	Insights tabs	report.view	Saved/custom/AI-generated reports
Build report	/insights/reports/new	Report Studio	report.create	Metric/group/filter/visualization builder
Report detail	/insights/reports/:reportId	Report Studio	report.view	View/edit/export/schedule report

Settings

Page	Route	Enter from	Permission	Purpose
Settings home	/settings	Sidebar	settings.view	Grouped business, access, operations, system and account settings
Business profile	/settings/business	Settings	settings.business	Identity, localization, contact
Locations	/settings/locations	Settings	settings.locations	Location directory
Create location	/settings/locations/new	Locations	settings.locations	Store/warehouse/stockroom creation
Location detail	/settings/locations/:locationId	Locations	settings.locations	Inventory, team, defaults and activity
Team	/settings/team	Settings	settings.team	Users, roles, locations and status
Invite user	/settings/team/invite	Team	settings.team	Email, role, location invitation
User detail	/settings/team/:userId	Team	settings.team	Assignments, role and activity
Roles & permissions	/settings/roles	Settings	settings.roles	System/custom roles
Role permission editor	/settings/roles/:roleId	Roles	settings.roles	Granular permissions and scopes
Taxes & currency	/settings/taxes-currency	Settings	settings.tax	Currencies, tax profiles, rounding
Document settings	/settings/documents	Settings	settings.documents	Invoice/receipt/PO/transfer numbering and templates
Security	/settings/security	Settings	settings.security	MFA, sessions, approvals, audit protections
Sales channels	/settings/sales-channels	Settings	settings.integrations	Store/marketplace channel connections
Channel detail	/settings/sales-channels/:channelId	Sales channels	settings.integrations	Sync direction, source of truth, activity
Integrations	/settings/integrations	Settings	settings.integrations	Commerce/accounting/payments/shipping/data
Integration detail	/settings/integrations/:integrationId	Integrations	settings.integrations	Authorization, sync configuration, activity
Notification settings	/settings/notifications	Settings	settings.notifications	Channel and cadence preferences

Page	Route	Enter from	Permission	Purpose
Barcode & printing	/settings/ barcode-printing	Settings	settings.printing	Barcode defaults, SKU generation, printers
Inventory rules	/settings/ inventory	Settings	settings.inventory	Negative stock, thresholds, counts, approvals
Purchasing settings	/settings/ purchasing	Settings	settings.purchasing	Defaults, approvals, receiving, AI purchasing
Transfer settings	/settings/ transfers	Settings	settings.transfers	Approvals, states, discrepancies, AI transfers
Import / export	/settings/import-export	Settings	settings.import_export	Data jobs and history
API & webhooks	/settings/api-webhooks	Settings	settings.api	API keys, events, delivery history
Audit log	/settings/audit	Settings	audit.view	Who/what/when/where/ before/after/source
Preferences	/settings/ preferences	Settings	settings.preferences	Appearance, density, language, accessibility
ThreadStock AI settings	/settings/ai	Settings	settings.ai	Recommendations, forecasting, prepared actions, auto-execution
Plan & usage	/settings/ subscription	Settings	billing.view	Plan limits and usage
Billing	/settings/billing	Settings	billing.manage	Payment method, billing address, invoices
Business data	/settings/data	Settings	settings.data	Export, retention, archive, closure

Help

Page	Route	Enter from	Permission	Purpose
Help & support	/help	Profile / Settings	Authenticated	Guides, support, feedback
System status	/help/status	Help	Authenticated	Service and sync status

Core end-to-end journeys

New business

Welcome -> Business -> Location -> Commerce -> Inventory start -> Team -> Complete -> Overview

After setup, route the user into the selected inventory starting flow rather than forcing inventory setup before Team.

Sell an item

Home/Sales -> New Sale -> Scan/Search -> Variant -> Cart -> Customer optional -> Checkout -> Payment -> Sale complete -> Receipt

A completed sale creates financial records, sale lines, payment state, inventory ledger events and audit events atomically/idempotently.

Replenish low stock

Overview/AI/Forecast -> Recommendation -> Review -> Edit -> Approve -> Draft/Create PO -> Supplier confirmation -> Receiving -> Inventory updated

Purchase receiving

Purchasing -> PO -> Receive -> Scan/enter quantities -> damaged/missing handling -> Complete partial/full receipt -> Inventory ledger -> PO state update

Store transfer

Product/AI/Transfers -> Create -> Approval (if needed) -> Picking -> Dispatch -> In Transit -> Receive -> discrepancy handling -> Completed

Stock count

Inventory -> Stock Counts -> Start -> Scan/manual count -> Variances -> Recount/Accept -> Approval -> Ledger reconciliation -> Completed

AI business action

Detect -> Explain -> Prepare -> Await approval -> Human edit/approve -> Execute -> Audit -> Surface outcome

Automation

Describe rule -> Build transparent conditions -> Test simulation -> Enable -> Run -> No action / Prepared action / Needs approval / Failure -> History

Offline

Connection lost -> Supported operation stored locally -> Outbox -> Connection restored -> Idempotent sync -> Success OR business conflict -> User resolves conflict

Page behavior contract

Every page specification should answer:

1. Who can access it?
2. What is the primary job-to-be-done?
3. What is the primary action?
4. What is visible before scrolling?
5. What happens on success?
6. What can fail?
7. What happens offline?
8. What is audited?
9. What is different on mobile vs desktop?
10. Where does Back return the user?

ThreadStock Detailed Module Specifications

Overview

Purpose: a calm command center, not a wall of charts. The first viewport should show business pulse, one high-value AI insight, needs-attention items and quick operational entry points.

Desktop can show compact metrics and one or two restrained analytics modules. Mobile Home prioritizes Today's Sales, ThreadStock Intelligence, Needs Attention and quick actions (Scan, Sell, Receive, Count Stock).

Inventory

Core object hierarchy: Product/Style → Variant (color/size/material) → Inventory by Location.

Inventory Studio supports search, saved filters, bulk selection and inspector on desktop; mobile defaults to visual product rows with compact mode optional. Product detail uses fashion imagery and progressively disclosed tabs.

Inventory operations include adjustments, stock counts, reconciliation, labels, catalog setup and imports. Every stock-changing operation is auditable.

Sales

Sales supports quick sale, held sale, checkout, receipt, history, returns/exchanges and customers. Customer is optional for a normal sale. Variant lookup should be scan-first in store workflows. Payment uncertain state must prevent duplicate sale completion.

Purchasing

Purchase Orders link supplier → destination → variants → quantity → costs → expected dates. AI can suggest quantities but the user can edit. Receiving is a separate operation and supports partial/damaged/missing quantities.

Transfers

Transfers move stock between locations through explicit states. Source and destination both have operational evidence; dispatch and receive are separate commands. Discrepancies are records, not silent corrections.

AI Studio

AI Home shows proactive business intelligence before chat. Core areas: actions, history, forecasts and anomalies. AI results can deep-link into operational workflows. High-impact proposals show What, Why, Impact and review/approval state.

Automations

Automations use transparent trigger/conditions/action/scope/approval. Natural language is only an authoring convenience. Test mode simulates; it never writes operational data.

Insights

Insights include sales, inventory, dead stock, ageing, profitability, location comparison, supplier performance, forecast accuracy and Report Studio. Every insight should either inform or connect to a safe operational action.

Settings

Settings is grouped, searchable and progressively complex. Areas: business, locations, taxes/currency, documents, team/roles, security, channels/integrations, notifications, barcode/printing, inventory/purchasing/transfer rules, import/export, API/webhooks, audit, preferences, AI, subscription, billing and business data.

Onboarding

Onboarding is short: business → location → commerce → inventory start → team → complete. Team is optional. The selected inventory-start method should be remembered and launched after onboarding completion.

Global surfaces

Notification Center, Approval Center, Activity, Sync Activity, Profile, Help and Global Search/Command Palette are cross-domain surfaces. They deep-link to the exact entity/action rather than generic module homes.

ThreadStock Behavior Specification

1. Global behavior

Context preservation

When a user drills into detail and returns, preserve the originating list's search, filters, sort, saved view, scroll position and selected row where practical.

Feedback

- Instant operation: update state immediately, no spinner flash.
- Short operation: disable/mark the triggering control and show concise inline progress.
- Long operation: create a background job with status and notification; never force the user to stare at a loading page.
- Success: toast for small actions; dedicated confirmation state for sales, receiving and other major workflows.
- Failure: explain what did not happen and provide a recovery action.

Destructive actions

Never use only "Are you sure?". State the object and consequence. Irreversible or high-impact actions require explicit confirmation and appropriate permission.

2. Authentication and workspace entry

A session does not automatically mean workspace access. Entry requires:

1. valid session;
2. active business membership;
3. completed/allowed business setup;
4. active role/location scope.

If any check fails, route to a safe blocked/setup state rather than rendering stale business data.

3. Inventory

Product creation

Style is the parent product. Color/size/material combinations create variants. SKU/barcode generation can be assisted but must resolve uniqueness before save.

Adjustment

A stock adjustment requires location, variant, signed quantity/change semantics, reason, actor and audit record. Update ledger + balance atomically.

Count

Counts can be blind or visible. Reconciliation posts only after review/approval. A recount does not mutate stock until reconciliation completes.

Label printing

Label generation is read-only with respect to stock. Print failures must not affect inventory.

4. Sales

Cart

Adding a scanned variant increments its cart quantity unless business rules prevent it. Parent-style selection requires variant choice.

Checkout

Do not create a completed sale until payment/business completion rules are satisfied. If payment status is uncertain, enter an explicit pending/verification state and prevent duplicate completion.

Return/exchange

Returns reference the original sale when available. Exchange posts return and replacement stock effects as an auditable business transaction. Manual/unlinked return may require manager approval.

5. Purchasing

PO states are canonical and should not be inferred from UI text alone. Suggested state progression:

draft -> awaiting_approval -> ordered -> partially_received -> received

Terminal alternatives: cancelled.

Receiving supports damaged/missing/over-received policy. Damaged stock must not silently enter sellable available stock.

6. Transfers

Suggested progression:

draft -> awaiting_approval -> picking -> in_transit -> partially_received/received

Terminal alternative: `cancelled`.

Dispatch removes/commits source stock according to the chosen inventory model. Receiving creates destination ledger events. Discrepancies are explicit records, not hidden balance corrections.

7. AI Studio

AI begins with proactive business intelligence, not an empty chat box. Natural language is an input method; results should become structured business objects when possible.

For actionable recommendations show:

- What ThreadStock found/recommends.
- Why: concise source signals.
- Impact: stock/revenue/cost/coverage where meaningful.
- Confidence: only calibrated/meaningful categories.
- Action: review/prepare/approve.

Sensitive execution must honor `AI_SYSTEM.md` and `SECURITY.md`.

8. Automations

The natural-language description compiles into transparent editable rules. Always show trigger, conditions, action, scope and approval requirement before activation.

"Test" is simulation only; it must not execute business writes.

Run result categories:

- completed;
- no_action;
- needs_approval;
- failed.

9. Insights

Analytics pages answer: What happened? Why does it matter? What can I do? Use charts only when they clarify a pattern. An insight that suggests an operation should deep-link to a prepared, reviewable workflow.

10. Search and command palette

Global search returns entities, destinations and actions. Natural-language queries may invoke AI read tools. High-impact write actions entered through search still route through the same review/approval policy.

11. Offline and sync

Offline banner is subtle and non-blocking for supported workflows. A queued action is never labeled synced until the server confirms it. Conflict examples must show business context such as sales/receipts that changed the stock after the user went offline.

12. Permissions

Three UI outcomes:

- Hide: actor should not discover/use the capability.
- Disable with explanation: capability is relevant but currently unavailable.
- Request approval/access: workflow explicitly supports escalation.

Server authorization remains authoritative.

13. Accessibility and localization

- Keyboard focus remains visible on desktop.
- Reduced motion removes nonessential movement.
- Dynamic text/large translations must not clip primary actions.
- RTL reverses directional layout where appropriate.
- Formatting is locale-aware; internal numeric values are not stored as formatted strings.

Canonical Route Manifest

These paths are the application route vocabulary. Do not create aliases casually. If a route changes, update this file, deep links, tests and analytics together.

Route	Module	Page	Parent / Entry	Permission
/onboarding/welcome	Onboarding	Welcome	Entry	Public
/onboarding/business	Onboarding	Business setup	/onboarding/welcome	Public
/onboarding/location	Onboarding	Location setup	/onboarding/business	Public
/onboarding/commerce	Onboarding	Commerce setup	/onboarding/location	Public
/onboarding/inventory	Onboarding	Inventory starting point	/onboarding/commerce	Public
/onboarding/team	Onboarding	Team setup	/onboarding/inventory	Public
/onboarding/complete	Onboarding	Setup complete	/onboarding/team	Public
/overview	Overview	Overview dashboard	App root	Authenticated
/notifications	Global	Notification center	Header bell	Authenticated
/approvals	Global	Approval center	Header / Overview / AI	Permission: approvals.view
/activity	Global	Recent activity	Header / entity activity	Authenticated
/sync	Global	Sync activity	Profile / offline banner	Authenticated
/profile	Global	My profile	Profile menu	Authenticated
/inventory	Inventory	Inventory Studio	Sidebar	inventory.view
/inventory/products/new	Inventory	Create product	/inventory	inventory.product.create
/inventory/products/:product Id	Inventory	Product detail	/inventory	inventory.view
/inventory/products/:product Id/edit	Inventory	Edit product	Product detail	inventory.product.edit
/inventory/products/:product Id/variants	Inventory	Variant matrix	Product detail	inventory.product.edit
/inventory/products/:product Id/stock	Inventory	Product stock	Product detail	inventory.view

Route	Module	Page	Parent / Entry	Permission
/inventory/products/:productId/activity	Inventory	Product activity	Product detail	inventory.view
/inventory/counts	Inventory	Stock counts	Inventory tabs	inventory.count.view
/inventory/counts/new	Inventory	Start stock count	/inventory/counts	inventory.count.create
/inventory/counts/:countId	Inventory	Active stock count	/inventory/counts	inventory.count.execute
/inventory/counts/:countId/reconcile	Inventory	Count reconciliation	Count detail	inventory.count.approve
/inventory/adjustments	Inventory	Adjustment history	Inventory tabs	inventory.adjustment.view
/inventory/adjustments/new	Inventory	Stock adjustment	Product / Inventory	inventory.adjustment.create
/inventory/labels	Inventory	Barcode & labels	Inventory tabs	inventory.labels.print
/inventory/catalog/categories	Inventory	Categories	Catalog setup	inventory.catalog.manage
/inventory/catalog/collections	Inventory	Collections	Catalog setup	inventory.catalog.manage
/inventory/import	Inventory	Inventory import	Inventory actions	inventory.import
/inventory/import/:jobId	Inventory	Import review	/inventory/import	inventory.import
/sales	Sales	Sales overview	Sidebar	sales.view
/sales/new	Sales	Quick sale	/sales	sales.create
/sales/held	Sales	Held sales	Sales tabs	sales.create
/sales/:saleId	Sales	Sale detail	/sales	sales.view
/sales/:saleId/receipt	Sales	Receipt	Sale detail	sales.view
/sales/:saleId/return	Sales	Return / exchange	Sale detail	sales.return
/sales/customers	Sales	Customers	Sales tabs	customer.view
/sales/customers/new	Sales	Create customer	Customers / New sale	customer.create
/sales/customers/:customerId	Sales	Customer detail	Customers	customer.view

Route	Module	Page	Parent / Entry	Permission
/purchasing	Purchasing	Purchase orders	Sidebar	purchasing.view
/purchasing/orders/new	Purchasing	Create purchase order	/purchasing	purchasing.po.create
/purchasing/orders/:poId	Purchasing	Purchase order detail	/purchasing	purchasing.view
/purchasing/orders/:poId/edit	Purchasing	Edit purchase order	PO detail	purchasing.po.edit
/purchasing/orders/:poId/receive	Purchasing	Receive purchase order	PO detail / Receiving	purchasing.receive
/purchasing/suppliers	Purchasing	Suppliers	Purchasing tabs	supplier.view
/purchasing/suppliers/new	Purchasing	Create supplier	Suppliers	supplier.create
/purchasing/suppliers/:supplierId	Purchasing	Supplier detail	Suppliers	supplier.view
/purchasing/receiving	Purchasing	Receiving queue	Purchasing tabs	purchasing.receive
/purchasing/returns/new	Purchasing	Return to supplier	PO detail	purchasing.return
/transfers	Transfers	Transfers	Sidebar	transfer.view
/transfers/new	Transfers	Create transfer	/transfers	transfer.create
/transfers/:transferId	Transfers	Transfer detail	/transfers	transfer.view
/transfers/:transferId/dispatch	Transfers	Dispatch transfer	Transfer detail	transfer.dispatch
/transfers/:transferId/receive	Transfers	Receive transfer	Transfer detail / Incoming	transfer.receive
/ai	AI Studio	Intelligence home	Sidebar / mobile AI	ai.use
/ai/actions	AI Studio	AI action queue	AI tabs / approvals	ai.action.view
/ai/actions/:actionId	AI Studio	AI action detail	AI actions	ai.action.view
/ai/history	AI Studio	AI history	AI tabs	ai.history.view
/ai/forecasts	AI Studio	Forecast center	AI tabs	ai.forecast.view
/ai/forecasts/:variantId	AI Studio	Forecast detail	Forecast center	ai.forecast.view

Route	Module	Page	Parent / Entry	Permission
/ai/anomalies	AI Studio	Anomaly center	AI tabs	ai.anomaly.view
/ai/anomalies/:anomalyId	AI Studio	Anomaly investigation	Anomaly center	ai.anomaly.view
/automations	Automations	Automations	Sidebar	automation.view
/automations/new	Automations	Create automation	/automations	automation.create
/automations/:automationId	Automations	Automation detail	/automations	automation.view
/automations/:automationId/edit	Automations	Edit automation	Automation detail	automation.edit
/automations/runs	Automations	Run history	Automations tabs	automation.view
/automations/runs/:runId	Automations	Run detail	Run history	automation.view
/insights	Insights	Insights home	Sidebar	insights.view
/insights/sales	Insights	Sales analytics	Insights tabs	insights.sales
/insights/inventory	Insights	Inventory analytics	Insights tabs	insights.inventory
/insights/dead-stock	Insights	Dead stock	Inventory analytics	insights.inventory
/insights/stock-ageing	Insights	Stock ageing	Inventory analytics	insights.inventory
/insights/profitability	Insights	Profitability	Insights tabs	insights.profitability
/insights/locations	Insights	Location comparison	Insights tabs	insights.locations
/insights/suppliers	Insights	Supplier performance	Insights tabs	insights.suppliers
/insights/forecast-accuracy	Insights	Forecast accuracy	Insights tabs	insights.forecast
/insights/reports	Insights	Report Studio	Insights tabs	report.view
/insights/reports/new	Insights	Build report	Report Studio	report.create
/insights/reports/:reportId	Insights	Report detail	Report Studio	report.view
/settings	Settings	Settings home	Sidebar	settings.view
/settings/business	Settings	Business profile	Settings	settings.business

Route	Module	Page	Parent / Entry	Permission
/settings/locations	Settings	Locations	Settings	settings.locations
/settings/locations/new	Settings	Create location	Locations	settings.locations
/settings/locations/:locationId	Settings	Location detail	Locations	settings.locations
/settings/team	Settings	Team	Settings	settings.team
/settings/team/invite	Settings	Invite user	Team	settings.team
/settings/team/:userId	Settings	User detail	Team	settings.team
/settings/roles	Settings	Roles & permissions	Settings	settings.roles
/settings/roles/:roleId	Settings	Role permission editor	Roles	settings.roles
/settings/taxes-currency	Settings	Taxes & currency	Settings	settings.tax
/settings/documents	Settings	Document settings	Settings	settings.documents
/settings/security	Settings	Security	Settings	settings.security
/settings/sales-channels	Settings	Sales channels	Settings	settings.integrations
/settings/sales-channels/:channelId	Settings	Channel detail	Sales channels	settings.integrations
/settings/integrations	Settings	Integrations	Settings	settings.integrations
/settings/integrations/:integrationId	Settings	Integration detail	Integrations	settings.integrations
/settings/notifications	Settings	Notification settings	Settings	settings.notifications
/settings/barcode-printing	Settings	Barcode & printing	Settings	settings.printing
/settings/inventory	Settings	Inventory rules	Settings	settings.inventory
/settings/purchasing	Settings	Purchasing settings	Settings	settings.purchasing
/settings/transfers	Settings	Transfer settings	Settings	settings.transfers

Route	Module	Page	Parent / Entry	Permission
/settings/import-export	Settings	Import / export	Settings	settings.import_export
/settings/api-webhooks	Settings	API & webhooks	Settings	settings.api
/settings/audit	Settings	Audit log	Settings	audit.view
/settings/preferences	Settings	Preferences	Settings	settings.preferences
/settings/ai	Settings	ThreadStock AI settings	Settings	settings.ai
/settings/subscription	Settings	Plan & usage	Settings	billing.view
/settings/billing	Settings	Billing	Settings	billing.manage
/settings/data	Settings	Business data	Settings	settings.data
/help	Help	Help & support	Profile / Settings	Authenticated
/help/status	Help	System status	Help	Authenticated

Route naming rules

- Plural nouns represent collections: /inventory/counts .
- new is for creation: /transfers/new .
- Entity IDs are named by entity: :productId, :poId .
- Actions that deserve a durable/deep-linkable workflow are child paths: /transfers/:transferId/receive .
- Tabs that are merely presentation can remain query/state instead of extra routes; routes above are minimum canonical destinations.
- Never encode business IDs in public paths unless multi-business deep linking requires it; active business scope is session/app context and revalidated server-side.

ThreadStock Canonical Data Model

This is a logical model, not a finished migration. Names may evolve only through an explicit architecture decision; relationships and invariants should remain stable.

Identity and tenancy

businesses

Business/tenant root.

Key concepts: id, legal/display identity, default locale/currency/timezone, status, created_at.

locations

Stores, warehouses, stockrooms or offices. Belongs to business.

memberships

Connects auth user to business. Status controls active access.

roles / permissions / membership_role_assignments

Roles group permissions. Assignment may include location scope.

Catalog

products

Fashion style/parent record: name, category, brand, collection, season, department, description, primary media.

product_variants

Sellable/stockable SKU: product_id, color, size, material/attributes, sku, barcode, price/cost defaults, active state.

categories / collections / brands / attribute definitions

Reusable catalog organization.

product_media

Images/media with sort order and metadata.

Inventory

inventory_ledger

Append-only movement/event table.

Recommended fields:

- id
- business_id
- location_id
- variant_id
- event_type
- quantity_delta_available
- quantity_delta_committed
- quantity_delta_damaged
- source_type
- source_id
- reason_code
- actor_user_id nullable for system events
- source_actor (user , ai , automation , integration , import)
- client_operation_id/idempotency key
- occurred_at server time
- metadata versioned JSON for non-index-critical detail

inventory_balances

Current materialized balance per location/variant. Updated atomically with ledger operation or recomputed from ledger where needed.

stock_counts / stock_count_lines

Count header, scope, status, counters and submitted values.

stock_adjustments

Business-facing adjustment record referencing one or more ledger events.

Sales

sales

Transaction/order header: business/location/customer/cashier, totals, currency, status, timestamps.

sale_lines

Variant, quantity, unit price, discount, tax, line total.

payments

Payment attempts/completions; store provider tokens/references, never raw card secrets.

refunds / return_lines

Refund/return business records and inventory effects.

customers

Lightweight commerce customer profile. Avoid collecting unnecessary sensitive profile data.

held_sales

Draft carts intentionally parked for later resume.

Purchasing

suppliers

Contact, terms, currencies, lead-time/defaults and status.

purchase_orders / purchase_order_lines

PO header and variant quantities/costs.

purchase_receipts / purchase_receipt_lines

Each receipt event supports partial and damaged quantities.

supplier_returns

Return-to-supplier header/lines and expected credit reference.

Transfers

transfers / transfer_lines

Source/destination, state and requested/sent/received quantities.

transfer_shipments / transfer_receipts / transfer_discrepancies

Operational evidence for dispatch, receiving and mismatch handling.

AI

ai_interactions

User request/query metadata and structured response references. Do not store hidden chain-of-thought.

ai_insights

Detected business insight with scope, source data references, expiry and read/dismiss state.

ai_action_proposals

Structured proposed action with lifecycle status, affected entities, expected impact, approval requirement and idempotency key.

ai_action_reviews

Human edits/approve/reject with reason and actor.

Automations

automations

Versioned rule definition, scope, status, approval policy and schedule/trigger definition.

automation_runs

One execution evaluation with metrics and outcome.

automation_run_actions

Prepared/executed actions linked to proposals or domain commands.

Insights and reports

saved_reports / report_runs / report_schedules

User-defined report specification, generated run and scheduled delivery configuration.

Platform

notifications

In-app notification records with target actor, entity link and read state.

approval_requests

Optional normalized approval queue abstraction referencing domain entities/proposals.

audit_log

Append-only security/business audit for privileged changes: actor/source, entity, operation, before/after digest or structured values, business/location, timestamp, request/correlation ID.

integration_connections / integration_mappings / sync_checkpoints

External channel configuration and sync state. Secrets stay in server secret storage, not plain tables/client.

outbox (client-local, optionally server mirror)

Queued offline mutation envelopes.

Invariants

1. SKU uniqueness is scoped according to business policy and enforced in database.
2. Barcode uniqueness is enforced where ThreadStock is authoritative; collision must be resolved, never silently overwritten.
3. Stock-changing records always produce ledger evidence.
4. Money is stored in integer minor units or a deliberate exact numeric strategy; never floating-point doubles for authoritative totals.
5. Every operational record is tenant-scoped.
6. Soft-delete/archive policy must preserve audit references.
7. State transitions are validated server-side; client cannot jump arbitrary statuses.
8. AI/automation source actions are distinguishable from direct user actions.

ThreadStock State Machines

State transitions must be validated server-side. The UI should render from canonical state rather than infer state from counts alone.

Purchase order

```
draft
-> awaiting_approval
-> ordered
-> partially_received
-> received

any allowed pre-terminal state -> cancelled
```

`received` and `cancelled` are terminal for normal editing.

Transfer

```
draft
-> awaiting_approval
-> picking
-> in_transit
-> partially_received (optional)
-> received

allowed pre-terminal state -> cancelled
```

A discrepancy can coexist with received state as a linked record; do not invent ambiguous status strings per screen.

Stock count

```
draft -> in_progress -> review_required -> awaiting_approval -> completed
      -> in_progress (recount)
```

Alternative terminal: cancelled.

Sale

```
draft/held -> payment_pending -> completed
completed -> partially_refunded -> refunded
```

Cancellation/void semantics should be explicit and depend on payment/fiscal policy.

AI action proposal

```
detected -> proposed -> prepared -> awaiting_approval -> approved -> executing -> completed
                                         -> rejected
                                         -> expired
                                         -> failed
```

Automation

Configuration state: `draft -> active <-> paused -> archived`.

Run state: `queued -> running -> completed | no_action | needs_approval | failed`.

Import job

`uploaded -> mapping -> validating -> needs_review -> importing -> completed | failed`.

Integration sync

`idle -> syncing -> success | warning | failed` with independent connection status
`connected | paused | disconnected | auth_required`.

ThreadStock AI System

Product role

ThreadStock AI is an operational intelligence layer. Its purpose is to reduce manual work while preserving human control, permissions and auditability.

Capability tiers

1. Observe

Read permitted business data and identify patterns: stockout risk, dead stock, supplier delay, unusual adjustments, location imbalance.

2. Recommend

Explain a suggested business response and expected impact.

3. Prepare

Create a structured proposal/draft such as a PO, transfer or report without yet executing the sensitive business action.

4. Execute

Allowed only if the action is within the configured automation/approval policy and the current actor/system identity has the server-side permission. High-impact actions should normally require approval.

Proposal lifecycle

detected -> proposed -> prepared -> awaiting_approval -> approved -> executing -> completed

Other outcomes: rejected, failed, cancelled, expired.

A proposal contains:

- business/location scope;
- proposal type;
- affected entity IDs;
- structured quantities/amounts;
- source signals/data snapshot references;
- explanation summary;

- expected impact;
- confidence category if meaningful;
- required permission/approval rule;
- creator source (AI/automation/user-assisted);
- idempotency key;
- expiry/staleness rules.

Server boundary

The Flutter app never contains provider API secrets. AI requests flow through a server-side gateway that:

1. authenticates the ThreadStock user/session;
2. resolves membership + permission + location scope;
3. builds minimal context;
4. invokes allowed read/proposal tools;
5. validates structured model output;
6. persists the interaction/proposal;
7. returns safe UI data.

Tool policy

Tools are explicitly allow-listed. Read tools cannot mutate. Proposal tools create draft/proposal records. Execution tools verify approval + freshness + idempotency immediately before executing.

Never give the model direct unrestricted SQL or service-role access.

Freshness

Before approving/executing an inventory recommendation, revalidate important live facts such as current stock, incoming supply, supplier state and permissions. If the proposal is stale, require refresh/review instead of applying outdated quantities.

User experience

AI should show structured results, not a long paragraph when a business object is appropriate.

Example:

- 18 SKUs at risk

- 640 units recommended
- Rs 482,000 estimated cost
- 37 days projected coverage
- Why: demand +22%, lead time 14 days, coverage 11 days
- Review / Edit / Approve

AI history

Store safe business provenance, not hidden reasoning traces. Useful history includes user prompt, structured summary, tools/data sources referenced, proposal/action IDs, final human decision and execution outcome.

Automations

Natural language can author a rule, but the saved automation is a transparent deterministic/versioned structure. "Test" performs simulation only. AI may help explain failures but does not hide the actual trigger/condition/action definition.

Evaluation

Evaluate AI features on business usefulness, not prose quality alone:

- stockout-risk precision/recall where labels exist;
- forecast error;
- approved vs rejected proposal rate;
- manual edits before approval;
- false anomaly rate;
- supplier/transfer recommendation outcomes;
- time saved;
- financial/stock impact with careful attribution.

Never claim causation that cannot be supported by the available data.

ThreadStock Security and Authorization Contract

1. Trust boundaries

The Flutter client is untrusted for authorization. It may improve UX, but security enforcement lives in Supabase/Postgres/privileged server functions.

2. Tenant isolation

Every query/write must be constrained to the actor's active business membership. RLS policies must not depend on user-supplied tenant IDs alone.

Where location scope applies, enforce it server-side for reads/writes that expose or mutate location-specific data.

3. Permission model

Permissions should be action-oriented, for example:

- inventory.view
- inventory.product.create
- inventory.adjustment.create
- inventory.count.execute
- inventory.count.approve
- sales.create
- sales.return
- purchasing.po.create
- purchasing.po.approve
- purchasing.receive
- transfer.create
- transfer.dispatch
- transfer.receive
- ai.action.approve
- automation.create
- insights.profitability
- settings.roles

Do not overload UI role names as database authorization logic. Roles are templates/groupings of permissions.

4. Sensitive operations

Examples:

- refund;
- large stock adjustment;
- PO approval;
- transfer approval/dispatch;
- AI prepared financial action;
- role/security/integration change.

These can require stronger confirmation, step-up authentication or a second approver according to business policy.

5. Secrets

Never ship:

- Supabase service-role key;
- model provider API keys;
- integration client secrets;
- webhook signing secrets;
- private encryption keys

inside Flutter assets, source, desktop binaries or logs.

6. RLS

RLS is mandatory for operational tables exposed through Supabase APIs. Policies should be explicit for select/insert/update/delete and call reusable authorization functions carefully. Security-definer functions must be narrowly scoped and reviewed.

7. Server commands

Complex sensitive writes should use RPC/Edge Functions with:

- authenticated caller;
- permission check;
- tenant/location validation;
- state-transition validation;
- idempotency key;
- transaction;
- audit record;

- canonical response.

8. Audit

Audit privileged and business-sensitive changes with:

- actor/user or system source;
- business/location;
- operation;
- entity + ID;
- relevant before/after values or digest;
- reason where required;
- request/correlation ID;
- server timestamp.

AI and automation must appear as distinct sources and record the approving human when applicable.

9. Files

Storage buckets should be business-scoped with signed/authorized access. Validate file type and size. Do not trust filename extensions alone.

10. Payments

Prefer tokenized payment-provider flows. Do not store raw card credentials. ThreadStock sale state must distinguish payment pending, completed, failed and uncertain/provider-verification states.

11. Offline data

Cache only what is necessary for supported workflows. Use secure storage for secrets and local database protections appropriate to platform risk. Signing out should clear/revoke sensitive local session state without corrupting queued business operations; policy for unsynced data must be explicit.

12. Logs and analytics

Never log access tokens, passwords, provider secrets, full payment data or unnecessary customer personal information.

13. Security definition of done

A feature that changes authorization, stock, financial state, AI execution or integrations is not complete until server enforcement and negative tests exist.

ThreadStock Visual Identity Contract

Version: 1.1

Status: **canonical for implementation**

Reference date: 17 September 2026

This document translates the approved ThreadStock visual references into implementation rules for Flutter, Figma handoff, Cursor, Antigravity and VS Code/Copilot. The attached approved screenshots and brand assets are the visual source of truth. When a token and a screenshot appear to disagree, preserve the screenshot's hierarchy and update the centralized token - do not hardcode a one-off color inside a feature.

1. Brand character

ThreadStock is a luxury fashion operations product, not a generic SaaS dashboard. Its visual language is **editorial fashion + calm enterprise software**:

- warm silk/ivory environment;
- near-black ink typography;
- antique champagne-gold accents;
- editorial serif display typography paired with a neutral sans-serif UI family;
- generous negative space;
- fine warm borders rather than heavy shadows;
- fashion photography and fabric texture used selectively;
- restrained status colors;
- high-density data only where the task requires it.

The primary product name is **ThreadStock**. **Atelier** may appear only where an approved product design explicitly uses it as an experience/theme label. Do not rename the product to Atelier OS in code, routes, analytics, database entities or new copy unless product requirements explicitly change.

2. Canonical brand assets

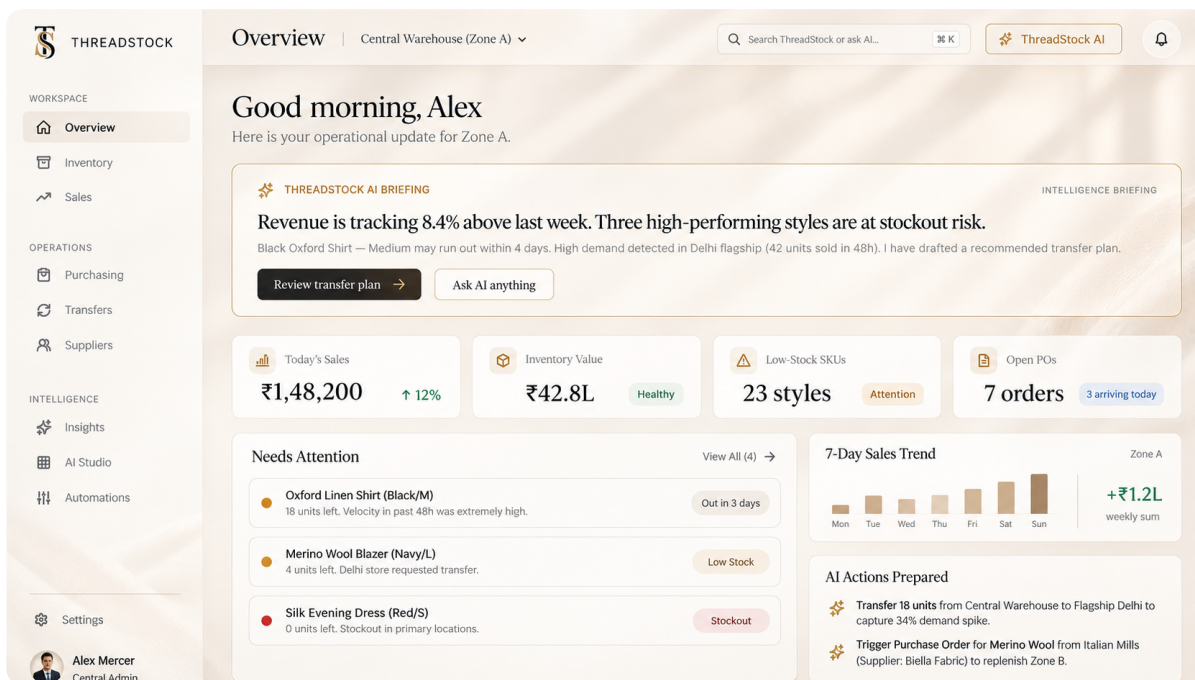
The blueprint pack includes:

- `assets/brand/threadstock-monogram.png` - compact TS monogram; use for app mark, splash mark and small brand placements.
- `assets/brand/threadstock-logo-lockup.png` - approved ThreadStock brand lockup reference.
- `assets/brand/threadstock-silk-background.png` - approved warm silk/fabric background.

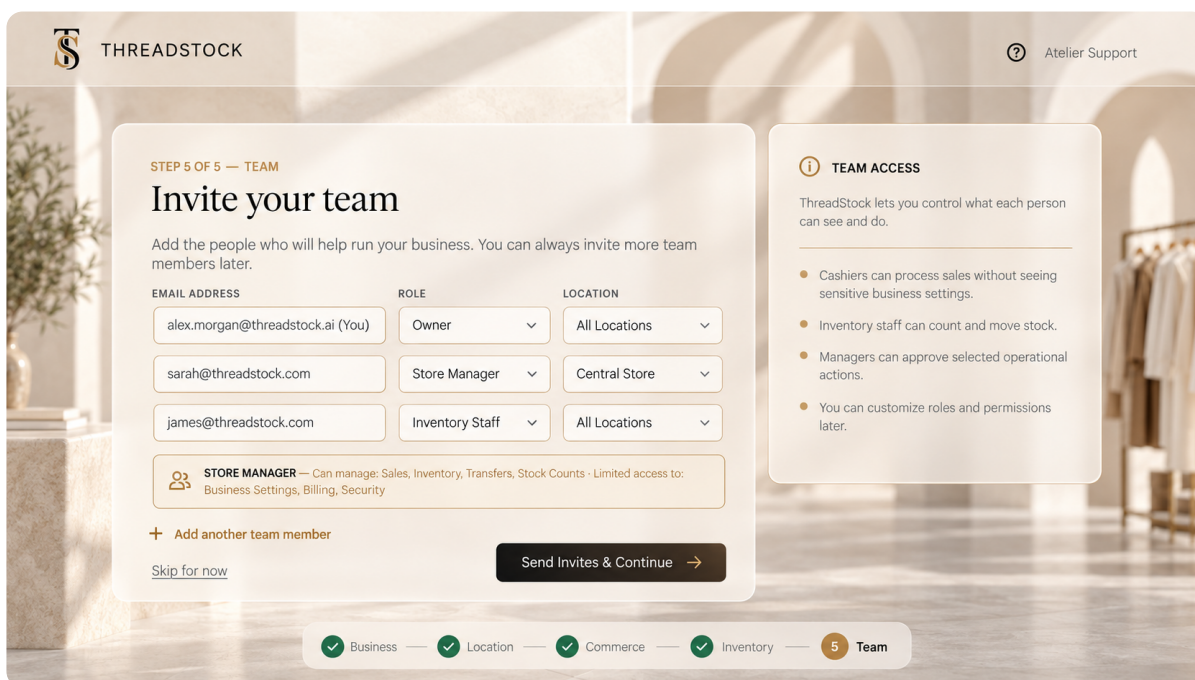
- `assets/reference/desktop-overview-reference.png` - approved desktop application visual reference.
- `assets/reference/onboarding-team-reference.png` - approved onboarding visual reference.

Production preference: export the logo from the final Figma source as SVG for scalable UI use. Until a vector asset is available, use the supplied raster asset rather than recreating the monogram with a font.

Approved visual references



Approved ThreadStock desktop Overview reference



Approved ThreadStock Team onboarding reference

Asset rules

- Never recolor the monogram with arbitrary theme colors.
- Do not crop through the TS mark.
- Keep a minimum clear space around the mark equal to roughly one quarter of the mark width.
- Use the monogram alone at compact sizes; use the full lockup only where the wordmark is readable.
- Do not place the logo over high-contrast image detail without a quiet surface/scrim.
- Do not recreate the wordmark with `Text()` in Flutter when the approved lockup asset is intended.

3. Color system

The palette below is calibrated to the approved screenshots and supplied logo assets. All feature code must consume semantic design tokens; raw hex values must not be scattered through widgets.

3.1 Light theme - approved ThreadStock Atelier palette

Semantic token	Hex	Usage
<code>brand.ink</code>	<code>#12120E</code>	Logo ink, strongest brand dark
<code>brand.gold</code>	<code>#B69450</code>	Primary antique-gold accent, icon accents, rules, selected emphasis
<code>brand.goldSoft</code>	<code>#D6BE89</code>	Soft champagne details, subtle decoration
<code>brand.bronzeText</code>	<code>#7A582B</code>	Accessible small gold/bronze text on light surfaces
<code>surface.canvas</code>	<code>#F3EBE4</code>	Ambient warm canvas / main shell background
<code>surface.shell</code>	<code>#FAF6F1</code>	Sidebar, header, quiet navigation surfaces
<code>surface.primary</code>	<code>#FFFDFB</code>	Cards, tables, forms, main content surfaces
<code>surface.secondary</code>	<code>#FBF6EF</code>	AI briefing, secondary panels, inset surfaces
<code>surface.selected</code>	<code>#F3ECE4</code>	Selected nav/row/tile background
<code>surface.input</code>	<code>#FFFCF8</code>	Inputs and form fields
<code>text.primary</code>	<code>#171513</code>	Main UI text
<code>text.secondary</code>	<code>#6B655E</code>	Body/supporting copy
<code>text.muted</code>	<code>#918981</code>	Metadata, quiet labels
<code>text.inverse</code>	<code>#FFFDFB</code>	Text on dark actions

Semantic token	Hex	Usage
<code>border.subtle</code>	<code>#E7DCD1</code>	Hairlines and section separators
<code>border.default</code>	<code>#D9C8B7</code>	Inputs and interactive outlines
<code>border.brand</code>	<code>#C9A56A</code>	Selected/AI/brand outline when needed
<code>action.primary</code>	<code>#272624</code>	Primary action button
<code>action.primaryPressed</code>	<code>#151412</code>	Pressed/strong action state
<code>ai.surface</code>	<code>#FBF2E5</code>	Intelligence surface
<code>ai.border</code>	<code>#D8B980</code>	AI/recommendation outline
<code>focus.ring</code>	<code>#8A642B</code>	Accessible keyboard focus ring

3.2 Semantic status colors

These are secondary to labels/icons. Never communicate status through color alone.

State	Text/Icon	Surface
Success / Healthy	<code>#1F6F46</code>	<code>#EDF5EF</code>
Warning / Attention	<code>#9A5D14</code>	<code>#F9EEE1</code>
Critical / Stockout	<code>#9D3F3B</code>	<code>#F7E7E4</code>
Information	<code>#3E5F8C</code>	<code>#EEF3F8</code>
Neutral	<code>#625D57</code>	<code>#F1EEEE</code>

3.3 Gold accessibility rule

`brand.gold #B69450` is a decorative/brand accent and does **not** have sufficient contrast for small body text on ivory/white. For small text, links, form labels or focus treatment use `brand.bronzeText #7A582B` or another tested accessible token. Never force the gold accent into text roles that reduce readability.

3.4 Dark companion theme

The approved references are light-first. Dark mode must preserve the same quiet luxury hierarchy rather than becoming black-and-gold gaming UI.

Semantic token	Hex
<code>surface.canvas</code>	<code>#0F0E0C</code>
<code>surface.shell</code>	<code>#14120F</code>
<code>surface.primary</code>	<code>#1B1814</code>
<code>surface.secondary</code>	<code>#211D18</code>
<code>surface.elevated</code>	<code>#29241E</code>

Semantic token	Hex
text.primary	#F7F0E8
text.secondary	#B2A99F
text.muted	#817970
border.subtle	#302A23
border.default	#463A2E
brand.gold	#D6BE89
brand.bronzeText	#E0C998
action.primary	#F2E8DE
action.onPrimary	#171513

4. Typography

ThreadStock deliberately uses **two typography voices**.

4.1 Editorial/display family

Preferred implementation family: **Cormorant Garamond**.

Use for:

- onboarding hero titles;
- desktop page titles such as `Overview`;
- major greetings (`Good morning, Alex`);
- large financial/stock metrics when editorial emphasis is desired;
- premium section titles in low-density surfaces.

Do not use the serif family for tables, form labels, SKU data, filters, dense lists or long operational paragraphs.

Recommended styles:

Style	Size / line	Weight	Usage
Display XL	48 / 52	600	Welcome/onboarding hero
Display L	40 / 44	600	Desktop greeting/hero
H1 Editorial	32 / 36	600	Major page title
H2 Editorial	26 / 31	600	Major section title
H3 Editorial	22 / 27	600	Premium panel title
Metric Editorial	34 / 38	600	Revenue, inventory value, major totals

Style	Size / line	Weight	Usage
Mobile Display	32 / 38	600	Mobile hero/title when appropriate

4.2 UI family

Preferred implementation family: **Inter**.

Use for:

- navigation;
- buttons;
- body copy;
- inputs and labels;
- tables;
- filters;
- chips/statuses;
- SKU/variant data;
- dialogs and system feedback.

Recommended styles:

Style	Size / line	Weight
UI Title	20 / 26	600
Section	16 / 22	600
Body Large	16 / 24	400
Body	14 / 20	400
Body Medium	14 / 20	500
Table	13 / 18	400
Label	12 / 16	600
Caption	12 / 16	400
Micro	11 / 14	500
Button	14 / 18	600

Uppercase micro labels such as **THREADSTOCK AI BRIEFING** or onboarding step labels may use Inter 600 with approximately `0.04em` tracking.

4.3 Brand wordmark

Do not infer the wordmark font. The approved logo lockup is an asset. When the full logo is not used and plain product-name text is required in navigation, use Inter 600/700 with restrained tracking rather than attempting to imitate the logo artwork.

4.4 Font loading

- Bundle/pin approved font files or package versions; avoid runtime font downloads in production-critical surfaces.
- If Cormorant Garamond is unavailable during boot, use a metrically reasonable serif fallback without changing layout drastically.
- Keep Inter as the operational UI fallback before platform default.
- Do not ship different fonts on Windows and macOS simply because they are available on the platform; preserve ThreadStock identity.

5. Background and surface treatment

The silk asset is part of the ThreadStock identity but must not impair productivity.

Onboarding, authentication and empty-business states

- Use `threadstock-silk-background.png` full-bleed with `BoxFit.cover`.
- Add a warm ivory scrim only as needed for predictable text contrast.
- Approved setup panels may use a softly translucent ivory surface (`surface.primary` at roughly 90-96% opacity) with a 1px warm border.
- A restrained backdrop blur (approximately 8-12 px) is acceptable on onboarding hero/setup panels only when performance is verified.

Main application shell

- The silk texture may appear as a **quiet ambient layer**, not as a competing background.
- Apply an ivory scrim equivalent to roughly 82-90% opacity over the image before placing operational content.
- Tables, forms and data panels remain near-opaque (`surface.primary`) for readability.
- Do not put dense tables directly on a photographic/fabric background.

Image intensity hierarchy

1. Onboarding/auth: strongest fabric presence.
2. Overview/empty state: subtle ambient fabric presence.
3. Inventory/Sales/Purchasing/Transfers/Settings: nearly flat warm canvas; texture is faint or absent behind data.
4. Scanner/camera: functional camera surface overrides brand background.

6. Surface geometry

- Standard card/panel radius: 12-14 px desktop.
- Onboarding/setup panel radius: 16-18 px.

- Mobile bottom sheet top radius: 20-24 px.
- Inputs: 8-10 px.
- Pills/chips: fully rounded only for short statuses/toggles, not every container.
- Default border: 1 px.
- Shadows: very soft or none. Prefer surface separation + warm border.

Do not make every section a card. The approved Overview uses cards for meaningful business objects while larger hierarchy comes from spacing and typography.

7. Primary actions

Primary buttons use near-black (`action.primary`) with warm-white text. Gold is not the default fill for primary buttons.

Gold is reserved for:

- AI/intelligence mark;
- selected accents;
- fine borders;
- arrow/icon detail inside premium actions;
- step indicators;
- small brand moments.

Secondary actions use a light surface and warm border. Tertiary actions are text/ghost controls.

8. Navigation

Desktop

- Warm quiet sidebar surface.
- Selected navigation uses a subtle warm selected background; avoid a large gold fill.
- Primary application sections remain: Overview, Inventory, Sales, Purchasing, Transfers, AI Studio, Automations, Insights, Settings.
- Supplier/other subpages remain contextual according to the product blueprint; do not add new top-level destinations solely because a screenshot includes them.

Mobile

Primary navigation remains `Home / Inventory / Scan / Sales / AI`.

Do not reproduce desktop sidebar hierarchy on phones.

9. AI visual language

AI must look native to ThreadStock, not like a generic chatbot brand.

- Use the antique-gold intelligence icon/accent.
- Use `ai.surface` + `ai.border` for briefings and prepared actions.
- Large AI text remains `text.primary`; do not make paragraphs gold.
- Avoid purple/blue AI gradients, neon glow, robot icons and excessive sparkles.
- The approved Overview `ThreadStock AI Briefing` pattern is the reference: editorial headline + concise evidence + action.

10. Charts and data visualization

- Primary series: antique gold/bronze.
- Secondary/reference series: warm graphite/gray.
- Positive financial delta: restrained green.
- Critical exceptions: restrained red only where required.
- Avoid rainbow charts.
- Keep gridlines very faint.
- Serif can be used for a major metric adjacent to a chart; axes, tooltips and labels remain Inter.

11. Form and onboarding style

Approved onboarding uses:

- full-bleed silk environment;
- editorial serif title;
- uppercase bronze step label;
- warm translucent/near-opaque setup panel;
- compact right-side explanation panel on desktop when useful;
- 5-step progress treatment with green completed states and antique-gold current step;
- near-black primary CTA;
- clear, sparse form controls.

Onboarding steps must remain operationally simple: Business → Location → Commerce → Inventory → Team.

12. Flutter implementation contract

Centralize theme implementation under the design system. Suggested names:

```
TsColors  
TsTextStyles  
TsSpacing  
TsRadius  
TsBorders  
TsShadows  
TsAssets  
TsTheme
```

Example semantic mappings:

```
TsColors.brandInk      -> #12120E  
TsColors.brandGold     -> #B69450  
TsColors.brandBronzeText -> #7A582B  
TsColors.canvas        -> #F3EBE4  
TsColors.shell         -> #FAF6F1  
TsColors.surface       -> #FFFDFB  
TsColors.surfaceSecondary -> #FBF6EF  
TsColors.textPrimary   -> #171513  
TsColors.textSecondary -> #6B655E  
TsColors.borderSubtle  -> #E7DCD1  
TsColors.primaryAction -> #272624
```

Recommended font constants:

```
TsFonts.display = Cormorant Garamond  
TsFonts.ui      = Inter
```

Feature widgets must not contain private color constants that reproduce these values.

13. Visual QA checklist

Before a UI task is complete, verify:

- The supplied ThreadStock logo asset is used correctly.
- Display/editorial text uses the serif family only in approved roles.
- Operational text uses Inter.
- No bright yellow/generic gold has replaced antique gold.
- Small gold text has accessible contrast (use bronze text token).
- Silk background intensity matches the screen type.
- Data surfaces remain readable and nearly opaque.
- Borders are warm and quiet, not cool gray.
- Primary actions remain near-black.
- AI uses native ThreadStock gold/surface treatment.
- Status colors are restrained and include text/icon labels.
- Light/dark themes use semantic tokens, not duplicated hex values.

- Mobile and desktop preserve the same identity without sharing identical layouts.

ThreadStock Design System Engineering Contract

Version: 1.1

The detailed visual identity is defined in [VISUAL_IDENTITY.md](#). The approved brand assets and reference screenshots under `assets/` are part of the implementation contract.

Product character

ThreadStock is **editorial fashion + calm enterprise software**: warm silk/ivory environments, near-black ink typography, antique champagne accents, an editorial serif display voice, neutral sans-serif operational UI and minimal decoration.

Do not reinterpret ThreadStock as a generic beige SaaS theme. The fabric background, logo, editorial typography and antique-gold detailing are deliberate brand elements.

Canonical visual tokens

All UI must consume semantic tokens. Do not hardcode visual values inside feature widgets.

Light

- Brand ink: `#12120E`
- Antique gold: `#B69450`
- Soft champagne: `#D6BE89`
- Accessible bronze text: `#7A582B`
- Canvas: `#F3EBE4`
- Shell/sidebar: `#FAF6F1`
- Primary surface: `#FFFDFB`
- Secondary/AI surface: `#FBF6EF`
- Primary text: `#171513`
- Secondary text: `#6B655E`
- Muted text: `#918981`
- Subtle border: `#E7DCD1`
- Default interactive border: `#D9C8B7`
- Primary action: `#272624`
- AI border: `#D8B980`

Dark companion

- Canvas: #0F0E0C
- Shell: #14120F
- Primary surface: #1B1814
- Secondary surface: #211D18
- Elevated surface: #29241E
- Primary text: #F7F0E8
- Secondary text: #B2A99F
- Muted text: #817970
- Subtle border: #302A23
- Antique gold: #D6BE89

Semantic status colors are restrained and must not be the only status signal.

Typography

ThreadStock uses two typography systems:

- **Cormorant Garamond** - editorial/display typography for onboarding hero titles, major page titles, greetings and selected high-level metrics.
- **Inter** - operational UI typography for navigation, body copy, forms, buttons, tables, filters, labels and dense data.

Do not use the serif family inside dense data tables or operational form controls. Do not recreate the brand wordmark with a font when the approved logo asset should be used.

Brand assets

Use the assets in `assets/brand/`:

- `threadstock-monogram.png`
- `threadstock-logo-lockup.png`
- `threadstock-silk-background.png`

Prefer the final Figma SVG export for production logo rendering when available. The supplied assets are the current visual reference.

Background behavior

- Onboarding/authentication: full-bleed silk background with warm contrast scrim as needed.
- Overview/empty states: subtle ambient silk background is allowed.

- Data-heavy pages: use mostly flat/near-opaque warm surfaces; fabric texture must not compete with tables/forms.
- Scanner: camera view is functional and takes precedence over brand background.

Avoid applying the fabric background uniformly at full intensity to every page.

Layout rules

Desktop

- Persistent compact sidebar and stable top bar.
- Main workspace uses hierarchy and spacing first.
- Cards/panels exist for real business objects, not every section.
- Data-heavy modules use professional tables and a contextual right inspector.
- Bulk actions appear when rows are selected.
- Keyboard navigation and visible focus are required.

Mobile

Primary navigation: [Home](#) / [Inventory](#) / [Scan](#) / [Sales](#) / [AI](#).

- One primary story per screen.
- Bottom sheets for selection, filters and lightweight edits.
- Full-screen pages for substantial workflows.
- Scanner is a first-class interaction surface.
- Never shrink desktop tables into phones; convert them to entity lists or grouped variant views.

Spacing and geometry

Use an 8-point foundation with 4-point micro spacing. Prefer 16/24/32 for section rhythm.

- Standard desktop panel radius: 12-14 px.
- Onboarding/setup panel: 16-18 px.
- Input radius: 8-10 px.
- Mobile sheet top radius: 20-24 px.
- Default border: 1 px.
- Shadows are restrained; prefer warm borders and surface separation.

Primary actions

Primary buttons use near-black `#272624` with warm-white text. Gold is an accent, not the default primary button fill.

Use antique gold for selected details, AI/intelligence marks, fine borders, progress steps and small premium emphasis.

Components

Create/reuse canonical components for:

- buttons (primary, secondary, tertiary, ghost, danger);
- text/search/number/currency fields;
- selects and mobile selection sheets;
- tabs/segments/filter chips;
- status label;
- product thumbnail/row;
- color swatch/size chip/variant cell;
- data table + toolbar;
- mobile entity rows;
- metric display;
- AI insight/action/review;
- approval row;
- timeline/activity row;
- right inspector;
- bottom sheet/modal/drawer;
- toast/banner/empty/loading/error/offline states.

Do not create a one-off component if a documented component can be extended safely.

AI visual language

AI uses the ThreadStock intelligence mark, antique-gold accent and warm quiet surface. Do not introduce purple/blue AI gradients, robot icons or excessive sparkles. Use large ink text for AI conclusions and gold for the intelligence signature rather than coloring whole paragraphs gold.

Motion

- Fast: ~120-160 ms for press/selection/quantity changes.
- Standard: ~180-240 ms for popovers/inspectors/tabs.
- Slow: ~280-360 ms for large sheets/page transitions.

No playful bounce by default. Honor reduced-motion preference.

Accessibility

- Target WCAG 2.2 AA contrast.
- #B69450 is decorative and not suitable for small text on light surfaces; use accessible bronze text #7A582B for small accent text.
- Do not use color alone for status.
- Maintain comfortable mobile touch targets.
- All icon-only actions need accessible labels/tooltips.
- Desktop must be keyboard navigable.
- Layouts must tolerate text scaling, long translations and RTL.

Internationalization

Never hardcode currency/date/number formatting. Support locale-aware currency, Indian and western grouping, decimal conventions, 12/24-hour time, RTL, time zones and country-specific address/tax structures.

ThreadStock Testing Strategy

Goals

Protect stock correctness, money correctness, tenant isolation, permissions, offline recovery and critical user journeys.

1. Static gates

- `dart format` / formatting check.
- `flutter analyze` with no ignored new warnings.
- generated-code consistency if code generation is used.
- migration lint/SQL verification.

2. Domain unit tests

Test business rules without Flutter/Supabase where possible:

- inventory quantity/event calculations;
- PO/transfer/sale state transitions;
- tax/discount/money arithmetic;
- permission decisions;
- forecast/action proposal transformations;
- automation rule evaluation.

3. Data/repository tests

Verify DTO mapping, pagination, retry/idempotency behavior and local/remote merge logic.

4. Database tests

For every sensitive schema/RLS/RPC change test:

- allowed tenant action;
- cross-tenant denial;
- location-scope denial;
- missing permission denial;
- invalid state transition denial;
- idempotent repeat call;
- ledger/balance consistency;

- audit creation.

5. Widget tests

At minimum cover:

- loading;
- populated;
- empty;
- error;
- permission denied;
- disabled/approval-required action;
- light/dark where visual logic differs.

6. Integration journeys

Critical automation targets:

1. onboarding → workspace;
2. create product + variants;
3. quick sale → payment success → stock reduction;
4. return/exchange;
5. create PO → partial receive → full receive;
6. create/dispatch/receive transfer with discrepancy;
7. stock count → variance → reconciliation;
8. AI proposal → edit → approve → domain action;
9. automation simulation → activation → run → approval;
10. offline sale/count → reconnect → sync/conflict.

7. Golden/visual testing

Use selectively for design-system primitives and high-value responsive shells, not every screen. Validate phone and desktop breakpoints.

8. Performance

Profile large catalog/list use cases. Use virtualization/pagination, image caching and query indexes. Establish budgets for cold start, inventory search response and scanner-to-result latency.

9. Regression policy

Any production bug involving stock, money, permissions or sync must receive a regression test before closure unless technically impossible and explicitly documented.

ThreadStock Implementation Roadmap

The design is broad; implementation should be vertical and testable. Do not build every screen shell first and wire behavior later.

Phase 0 - Foundation

- repository/tooling;
- environments and Supabase projects;
- app bootstrap/config validation;
- design tokens/components;
- typed routing;
- localization/formatting;
- authentication and business context;
- permission framework;
- logging/error model;
- local database + sync/outbox skeleton.

Exit: authenticated user can enter an authorized empty workspace on mobile and desktop with no legacy/fake data.

Phase 1 - Catalog and inventory core

- product/style/variant model;
- inventory ledger/balance;
- inventory list/detail;
- create/edit product;
- stock adjustment;
- barcode scanning lookup;
- labels;
- categories/collections;
- import foundation.

Exit: real stock can be created, viewed and adjusted with audit evidence.

Phase 2 - Sales

- quick sale;
- held sale;
- checkout/payment abstraction;
- receipts;

- sales history/detail;
- returns/exchanges;
- customers.

Exit: sale changes inventory atomically and can be safely returned.

Phase 3 - Purchasing and receiving

- suppliers;
- PO create/detail/approval;
- receiving full/partial/damaged;
- supplier return;
- incoming inventory views.

Exit: buy-to-receive loop is production-ready.

Phase 4 - Transfers and stock counts

- transfer lifecycle;
- dispatch/receive/discrepancy;
- stock count/blind count;
- reconciliation/approval.

Exit: multi-location stock operations are trustworthy.

Phase 5 - Insights foundation

- event/read models;
- sales/inventory analytics;
- dead stock/ageing;
- location and supplier performance;
- report export.

Exit: analytics are based on real operational data, not placeholder charts.

Phase 6 - AI read intelligence

- AI gateway;
- natural-language search;
- stockout/dead-stock/supplier/location insights;
- forecast center;
- anomaly detection;

- AI history/provenance.

Exit: AI produces useful read-only intelligence with evaluation telemetry.

Phase 7 - AI prepared actions and approvals

- action proposal schema;
- replenishment PO proposals;
- transfer proposals;
- review/edit/approve/reject;
- approval center;
- execution revalidation + audit.

Exit: AI can safely prepare real work without bypassing human/permission controls.

Phase 8 - Automations

- rule model;
- natural-language authoring;
- deterministic simulation;
- scheduler/event triggers;
- run history/failures;
- approval integration.

Phase 9 - Integrations and enterprise settings

- channels/integrations framework;
- teams/roles refinements;
- API/webhooks;
- advanced tax/doc settings;
- plan/billing as needed.

Phase 10 - Production hardening

- offline conflict scenarios;
- accessibility audit;
- performance/load testing;
- security review;
- backup/restore/export procedures;
- observability dashboards;
- store release/desktop packaging;

- migration and incident runbooks.

Delivery rule

Each phase should ship vertical behavior across UI → domain → server → tests. A completed Figma page without real domain behavior is not a completed product feature.

ThreadStock Definition of Done

A feature is done only when applicable items below are satisfied.

Product

- Canonical route and entry/exit path are defined.
- Primary and secondary actions are real.
- No placeholder buttons or fake data behavior remain.
- Mobile and desktop behavior are intentionally specified where applicable.

UX

- Loading, empty, error and permission states exist.
- Offline behavior is explicit for operational features.
- Back/context preservation works.
- Light/dark theme are correct.
- Accessibility labels/focus/touch targets are present.
- Localization-safe formatting is used.

Domain/data

- Business invariant is enforced in domain/server layer.
- Sensitive multi-record write is transactional.
- Idempotency exists for retryable critical commands.
- Inventory-changing operations create ledger evidence.
- Audit record exists where required.

Security

- RLS/server permission enforcement is implemented.
- Negative permission and cross-tenant tests pass.
- No secrets exist in client source/assets/logs.

Quality

- Formatting/analyzer pass.
- Relevant unit/widget/integration tests pass.
- Database verification tests pass where changed.
- No unresolved TODO that changes correctness.

- Errors are user-safe and developer diagnostics are observable.

Documentation

- Route/data/status vocabulary changes are updated in blueprint docs.
- Migration and operational implications are documented.
- New reusable component is added to design system rather than copied.

AI Coding Platform Instructions

The pack includes repository-native instruction files for Cursor, Google Antigravity and VS Code/GitHub Copilot, plus `AGENTS.md` as a cross-agent source.

Cursor

Project rules live in `.cursor/rules/*.mdc`.

Included:

- `.cursor/rules/00-threadstock-core.mdc`
- `.cursor/rules/10-flutter-ui.mdc`
- `.cursor/rules/20-supabase-data.mdc`
- `.cursor/rules/30-quality.mdc`

Keep rules version-controlled. `AGENTS.md` is also included as a plain-Markdown agent instruction source.

Google Antigravity

Workspace rules live under `.agents/rules/`.

Included:

- `.agents/rules/threadstock-core.md`
- `.agents/rules/threadstock-ui.md`
- `.agents/rules/threadstock-data.md`

If you maintain personal global Antigravity rules, keep them generic; project behavior belongs in the workspace repository.

VS Code / GitHub Copilot

Repository-wide instructions:

- `.github/copilot-instructions.md`

Path-specific instructions:

- `.github/instructions/flutter.instructions.md`
- `.github/instructions/supabase.instructions.md`
- `.github/instructions/testing.instructions.md`

`AGENTS.md` provides compatible project-level context for agent workflows that support it.

Rule precedence philosophy

Keep the most important invariant in `AGENTS.md` and platform core rules; keep path-specific implementation guidance close to the relevant file glob. Do not duplicate large architecture documents into every rule file; link agents to the canonical docs.

First prompt for a coding agent

Use:

Read `AGENTS.md`, `ARCHITECTURE.md`, `PRODUCT_BLUEPRINT.md`, `ROUTE_MANIFEST.md`, `VISUAL_IDENTITY.md`, `DESIGN_SYSTEM.md` and `SECURITY.md`. Then summarize the constraints relevant to the task before changing code. Do not implement placeholders. Build the smallest production-ready vertical slice with tests.

Visual implementation setup

For any UI task, agents must inspect `VISUAL_IDENTITY.md`, `DESIGN_SYSTEM.md` and the approved references under `assets/reference/` before generating new styling. Use the supplied assets under `assets/brand/` for the current logo/background references.

Implementation targets:

- display/editorial font family: Cormorant Garamond;
- operational UI font family: Inter;
- primary action: near-black, not gold;
- antique gold is a restrained brand/AI accent;
- light theme is the approved primary appearance;
- silk/fabric imagery is strong in onboarding and quiet behind operational data.

Do not embed arbitrary font binaries into agent output. Add approved licensed font assets to the application repository through the normal design/engineering asset pipeline, then pin them in Flutter configuration.

ThreadStock Engineering Skills

This file defines the specialist capabilities an AI coding agent should apply when implementing ThreadStock. A task can require multiple skills; choose the smallest set that safely completes the work.

1. Product workflow skill

Use for any user-facing feature. Before coding, identify:

- actor and role;
- starting page and canonical route;
- happy path;
- empty/loading/error/permission/offline states;
- audit consequence;
- mobile versus desktop presentation;
- completion signal and next action.

Never implement a screen without its operational exit path.

2. Flutter feature skill

Use feature-first structure:

```
lib/features/<feature>/  
  data/  
    datasources/  
    dto/  
    repositories/  
  domain/  
    entities/  
    repositories/  
    services/  
    usecases/  
  presentation/  
    controllers/  
    pages/  
    widgets/
```

Keep reusable design primitives in `lib/design_system/`, not copied into features.

Visual implementation contract:

- read `VISUAL_IDENTITY.md` before visual/UI work;

- use Cormorant Garamond only for approved editorial/display roles and Inter for operational UI;
- use semantic theme tokens instead of feature-local hex values;
- use approved `assets/brand/` logo/background assets;
- primary actions are near-black, while antique gold is a restrained accent;
- keep silk texture strong on onboarding and subtle/absent behind dense operational data.

3. Responsive/adaptive UI skill

ThreadStock uses intentional layouts, not scaled layouts.

- Phone: task-first lists, bottom sheets, bottom navigation, scanner-centric flows.
- Tablet: adaptive navigation and split views when useful.
- Desktop: sidebar, tables, keyboard shortcuts, bulk action bar, right inspector.
- Preserve user context when moving between list/detail/actions.

4. Supabase/Postgres skill

Use Postgres as the system of record. Prefer:

- relational constraints;
- foreign keys;
- transaction-safe RPCs for multi-table operations;
- RLS for tenant and role boundaries;
- append-only audit/ledger records for inventory and sensitive changes;
- database timestamps as authoritative server time.

Never let the client assemble a financially or inventory-sensitive multi-step transaction using unrelated writes when the database can perform it atomically.

5. Inventory accounting skill

Inventory quantity is derived from a ledger of business events. Typical event types:

- `opening_balance`
- `purchase_receipt`
- `sale`
- `sale_return`
- `transfer_dispatch`
- `transfer_receive`
- `stock_adjustment`

- stock_count_reconciliation
- supplier_return

Maintain available, committed, incoming and damaged semantics explicitly. Never conflate them.

6. Offline/sync skill

Use a local relational store for cached read models and an outbox for permitted offline writes.

Each queued mutation needs:

- client_operation_id UUID;
- business_id and location scope;
- entity type and operation;
- payload version;
- created_at_local;
- retry count/status;
- server result/version when synced.

Never silently resolve stock conflicts. Surface business-readable conflict context.

7. AI workflow skill

AI output that can cause a business change follows:

detected -> proposed -> prepared -> awaiting_approval -> approved -> executing -> completed

Alternate terminal states:

rejected | failed | cancelled | expired

AI can use tools only within the actor's permission and business/location scope. Store proposal inputs, model/tool version metadata, structured output and final human/action outcome.

8. Authorization skill

Permission checks exist at three layers:

1. UI visibility/enablement.
2. Domain/service guard for fast feedback.
3. Server/RLS/RPC enforcement as the security boundary.

Never rely on layer 1 or 2 alone.

9. Testing skill

Prioritize tests around money, stock and permission boundaries.

Minimum for a feature:

- domain unit tests;
- repository/data adapter tests;
- widget tests for main states;
- integration test for the critical journey;
- RLS/SQL tests if data access changes.

10. Migration skill

Every database migration must be:

- deterministic;
- environment-neutral;
- safe for existing rows;
- reviewed for RLS impact;
- accompanied by verification SQL;
- reversible where practical or explicitly documented when not.

11. Observability skill

Instrument meaningful operations, not every tap. Capture:

- operation name;
- actor/business/location scope (non-secret identifiers);
- duration;
- success/failure category;
- sync state;
- AI proposal/action IDs where relevant.

Never log secrets, full payment credentials or private access tokens.

12. Product copy skill

Use concise operational language: `Receive Stock`, `Create Transfer`, `Approve`, `Low Stock`.

Avoid technical or inflated copy such as `Execute Inventory Replenishment Procedure` Or consumer-AI language like `Working our magic`.